# Code-Layout, Readability and Reusability

| | |
|---|---|
| **Date** | 01July 2026 |
| **Team ID** | SWTID-2026-4350 |
| **Project Name** | Credit Card Approval Prediction |
| **Maximum Marks** | 5 Marks |

## Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

## Code Layout Checklist:

| S.No | Code Quality Parameter | Description | Followed (Yes / No / Partial) | Remarks |
|---|---|---|---|---|
| 1 | Consistent Indentation | Uniform spacing the code /tabs used throughout the code | Yes | Proper indentation is maintained throughout the Flask application and model definitions. |
| 2 | Proper File Structure | Files and folders are logically organized | Partial | Main application is organized, but supporting files (config.py, prediction.py, templates) are stored separately |
| 3 | Meaningful Variable Names | Variables reflect their purpose clearly | Yes | Variables such as ApplicantDetail, ApprovalPrediction, CreditHistory, and predictor clearly describe their purpose |
| 4 | Function / Method Names | Functions are descriptively named | Yes | Functions like initialize_database() and get_predictor() use descriptive names. |
| 5 | Code Comments | Inline and block comments explain logic | Partial | Logging and readable code improve understanding, though additional inline comments can enhance maintainability. |
| 6 | Modular Design | Code is split into reusable functions/ modules | Yes | Database models, prediction service, configuration, and web routes are separated into reusable modules. |
| 7 | No Redundant Code | Duplicate or unused code is removed | Yes | Code avoids unnecessary duplication by using reusable database models and prediction functions. |
| 8 | Error Handling | Exceptions and errors are handled gracefull | Partial | Logging is configured and exceptions are partially handled. More comprehensive exception handling can be added for database and prediction failure |

## Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology | Where Reused | Reusability Level (High / Medium / Low) |
|---|---|---|---|---|
| 1 | Flask Application (app.py) | Python, Flask | Main web application and route handling | High |
| 2 | SQLAlchemy Models | Python, Flask-SQLAlchemy | Database operations throughout the application | High |
| 3 | Credit Predictor (CreditPredictor) | Python, Machine Learning | Loan approval prediction service | High |
| 4 | Database Initialization | Python | Database setup and default data seeding | Medium |
| 5 | HTML Templates (home.html, index.html, result.html) | HTML, Jinja2 | User interface rendering | High |

### Overall Code Quality Assessment:

| Aspect | Rating (1-5) | Comments |
|---|---|---|
| Code Layout & Structure | 5 | Well-structured Flask application with clear separation of routes, models, and prediction logic. |
| Readability | 4 | Meaningful naming conventions and organized code improve readability. Additional comments can further enhance clarity. |
| Reusability | 5 | Modular architecture allows prediction logic, database models, and templates to be reused in future projects. |
| Documentation / Comments | 4 | Logging is available, but more detailed code comments and documentation would improve maintainability. |
| Overall Score | 4.5 | The project demonstrates good coding practices with modular design, reusable components, and a maintainable structure. |